

Modelos computacionales

Tenemos dos tipos:

- Modelos Generativos : Gramáticas
- Modelos Reconocedores: Autómatas

En este curso solo vamos a ver lenguajes regulares y sus dos modelos computacionales

- Autómatas finitos
- Gramáticas Regulares

Cadena & Alfabeto

Una **cadena**, o palabra, es una secuencia finita de símbolos unidos por concatenación. Esencialmente, es una serie de símbolos en un orden específico. Por ejemplo, "aaabbbccc" es una cadena.

Un **alfabeto** es un conjunto finito de símbolos. Se suele usar para denotar alfabetos la letra griega Σ . Cualquier conjunto finito de símbolos puede ser un alfabeto, por ejemplo:

- $\Sigma_1 = \{a, b, c\}$
- $\Sigma_2 = \{0, 1\}$

Las cadenas se construyen utilizando los símbolos de un alfabeto específico. La cadena vacía, denotada como ϵ , es una cadena que no contiene ningún símbolo.

Algunos ejemplos de cadenas y alfabetos:

- La cadena "abbba" y el alfabeto $\{a, b, c\}$.
- La cadena "001000001" y el alfabeto binario $\{0, 1\}$.

Ten en cuenta que el **orden de los símbolos en una cadena es importante**, ya que "aba" es diferente a "aab".

Denotaremos con Σ^* al conjunto de todas las cadenas posibles del alfabeto Σ (el universo de cadenas del alfabeto). Y con ϵ denotaremos a la única cadena que no contiene símbolos, llamada cadena vacía.

Definición recursiva de lenguajes regulares

¿Qué es un lenguaje?

Un lenguaje en el alfabeto Σ es cualquier conjunto de cadenas de Σ

¿Qué es un lenguaje regular?

Los lenguajes regulares son aquellos que se pueden construir (recursivamente) a partir de ciertos lenguajes muy básicos o atómicos, aplicando una cantidad finita de ciertos operadores de lenguajes

La definición recursiva de **lenguajes regulares** puede parecer complicada al principio, pero es más fácil de entender si la desglosamos paso a paso. Imagina que estás construyendo una torre de bloques.

• **Empiezas con bloques básicos:** En el caso de los lenguajes regulares, estos bloques básicos son:

- El conjunto vacío ($\emptyset$), que no tiene ninguna cadena.
- El conjunto que solo contiene la cadena vacía ($\{\epsilon\}$), que es como un bloque invisible.
- Conjuntos que solo contienen un símbolo del alfabeto, como $\{a\}$, $\{b\}$, etc.

• **Luego, puedes combinar estos bloques básicos usando reglas:** Estas reglas te permiten construir lenguajes regulares más complejos a partir de los más simples:

◦ **Unión:** Puedes "pegar" dos lenguajes regulares para formar uno nuevo. Es como unir dos torres de bloques.¹

◦ **Concatenación:** Puedes unir dos lenguajes regulares uno después del otro. Es como poner una torre de bloques encima de otra.¹

Clausura de Kleene: Puedes tomar un lenguaje regular y repetirlo cualquier número de veces, incluso cero veces. Es como si tuvieras una máquina que puede duplicar tu torre de bloques tantas veces como quieras.

Finitud y Regularidad

- **Todo lenguaje finito es un lenguaje regular**

- Los lenguajes regulares son cerrados para los operadores de conjunto y la reversa

Teorema

Si $L_1, L_2 \in LR^\Sigma$, entonces:

1. $L_1 \cup L_2 \in LR^\Sigma$ (trivial por definición de lenguaje regular)
2. $L_1 \cap L_2 \in LR^\Sigma$
3. $L_1 - L_2 \in LR^\Sigma$
4. $\overline{L_1} \in LR^\Sigma$
5. $L_1^R \in LR^\Sigma$

Expresiones regulares?

Las expresiones regulares son expresiones o fórmulas que sirven para describir lenguajes regulares (de allí su nombre) en forma muy legible y compacta.

Resultado

Un lenguaje es regular si y sólo si es denotado por una expresión regular.

Autómatas Finitos

Un **autómata finito** es un modelo matemático usado para representar y manipular lenguajes regulares. Es una máquina abstracta con un número finito de estados, que procesa una cadena de entrada símbolo por símbolo y determina si esa cadena pertenece o no a un lenguaje.

(más en filminas de clase)

Tipos de AF

Determinísticos (AFD)

► No-Determinísticos (AFN) ►

No-Determinísticos con transiciones espontáneas (AFNε)

Lenguaje aceptado de un AFD

Definimos el lenguaje aceptado de M , denotado con $L(M)$, como el conjunto de todas cadenas aceptadas por M : $L(M) = \{\alpha \in \Sigma^* : \rightarrow \delta(q_0, \alpha) \in F\}$

Determinismo vs No-Determinismo

Aspecto	Determinista (AFD)	No Determinista (AFND)
Transiciones	Cada estado y símbolo tienen una única transición posible .	Puede haber varias transiciones para un mismo símbolo.
Simplicidad	Más fácil de implementar.	Más difícil de implementar directamente.
Eficiencia	Procesa una cadena en tiempo lineal.	Puede necesitar explorar múltiples caminos.
Equivalencia	Puede simularse con un AFND.	Puede convertirse en un AFD.
Transiciones vacías	No existen transiciones ϵ (sin consumir símbolos).	Puede incluir transiciones ϵ .

(Para evitar confusiones, denotaremos con Δ las funciones de transición no-deterministas, mientras que con δ las deterministas)

Resultado

Los AFD y los AFN son computacionalmente equivalentes.

Estados Alcanzables y solubles

En el contexto de autómatas finitos y lenguajes formales, los conceptos de estados alcanzables y solubles son importantes para optimizar y analizar la estructura del autómata. Aquí te explico cada uno:

1. Estados Alcanzables:

- Un estado $q \in Q$ es alcanzable si existe al menos un camino desde el estado inicial q_0 hasta q , utilizando las transiciones definidas por la función δ .
- Significado: Estos son los estados que efectivamente se pueden visitar durante la ejecución del autómata.
- Importancia:
 - Los estados no alcanzables son innecesarios, ya que nunca se utilizan al procesar una cadena.
 - Identificarlos ayuda a simplificar el autómata eliminándolos.

Ejemplo:

Supongamos un autómata con:

- $Q = \{q_0, q_1, q_2, q_3\}$
- q_0 es el estado inicial.
- Transiciones:
 - $\delta(q_0, a) = q_1$
 - $\delta(q_1, b) = q_2$

El estado q_3 es no alcanzable si no hay ninguna transición que lo conecte con el estado inicial q_0 .

2. Estados Solubles:

- Un estado $q \in Q$ es soluble si desde q es posible alcanzar algún estado de aceptación $q_f \in F$, utilizando las transiciones del autómata.
- Significado: Representan estados desde los cuales el autómata puede procesar una parte de la cadena y llegar a aceptar la cadena completa.
- Importancia:
 - Los estados no solubles no contribuyen a aceptar cadenas válidas, por lo que se pueden eliminar.
 - Ayudan a optimizar el autómata.

Ejemplo:

Supongamos un autómata con:

- $Q = \{q_0, q_1, q_2\}$
- $F = \{q_2\}$
- Transiciones:
 - $\delta(q_0, a) = q_1$
 - $\delta(q_1, b) = q_2$

Si q_2 es el único estado de aceptación, pero no hay un camino desde q_0 o q_1 hacia q_2 , entonces esos estados serían no solubles.

Al limpiar los estados no alcanzables y no solubles, se obtiene un autómata más simple y eficiente.

Determinizando un AFN

aplicamos el algoritmo y luego de eliminar los estados no alcanzables y no solubles obtenemos su AFD equivalente que acepta el mismo lenguaje

Transiciones Espontáneas

Una transición espontánea, si está definida, puede tener lugar como no (por eso su nombre), en consecuencia, es en sí misma una transición no-determinista.

► Se pueden pensar como una transición “gratis”, en el sentido de que el autómata cambia de estado sin procesar o consumir un símbolo de la cinta de entrada.

► Las denotaremos con el símbolo ϵ , así: $\Delta(q, \epsilon) = \{q_1, \dots, q_k\}$ significa que el autómata puede libremente transicionar de q a cualquier q_i sin leer símbolo alguno de la cinta de entrada (y sin desplazar su cabezal).

//////////

Un lenguaje es regular si y solo si es aceptado por un AF

Teorema kleene:  Lección 15: Teorema de Kleene (UACM - TC)